

micro-vLLM, CUDA Python, 하이브리드 프리필 디스패치, 프리픽스 KV 캐시, CUDA 그린 컨텍스트

Windows-Native LLM Serving System Implementation and Performance Analysis Case Study:

micro-vLLM, CUDA Python, Hybrid Prefill Dispatch, Prefix KV Cache, and CUDA Green Contexts

요약

고성능 LLM 서빙은 FlashAttention·Triton·NCCL 등 리눅스 중심 스택이라 학습자가 프리필·디코드·KV 캐시·스케줄러·메모리 할당 상호작용을 관찰하기 어렵다. 본 논문은 Windows 네이티브 CUDA Python/cuTile 경로의 소형 교육 아티팩트 마이크로-vLLM 구현·성능 분석 사례연구다. WSL2 FlashAttention 경로는 2138.55 tok/s로 초기 Windows cuTile 경로(463.35 tok/s) 대비 4.62배 빠르다. cuTile KV-스토어 수리와 하이브리드 프리필 디스패치로 eager-hybrid 경로는 557.66 tok/s(20.4% 향상)에 도달했고, CUDA 그래프 재생은 성능이 하락하는 부정적 결과를 보였다. 고정 컨텍스트 에이전트 워크로드에서 워 프리픽스 KV 캐시 재사용은 연산 프리필 토큰을 64로 줄이고 TTFT를 41.1%·66.1%·79.7% 감소시켰다(프리픽스 변경 음성대조 0% 히트). CUDA 그린 컨텍스트는 활성화 검증 후 보호 디코드 지연을 완화(부호검정 $p=0.0004$)하나 순차 프리필 중단은 제거하지 못한다. 이는 구현·측정·검정·부정 결과를 모두 고려한 서빙 교육의 근간이다.

키워드: LLM 서빙, CUDA Python, cuTile, 하이브리드 프리필 디스패치, 프리픽스 KV 캐시, CUDA 그린 컨텍스트

High-performance LLM serving relies on Linux-centric stacks (FlashAttention, Triton, NCCL, vLLM-style schedulers), making it hard for students and Windows-based local researchers to inspect how prefill, decode, KV-cache management, scheduling, and allocation interact inside an engine. We present micro-vLLM as a Windows-native CUDA Python/cuTile educational artifact. The WSL2 FlashAttention reference reaches 2138.55 tok/s versus 463.35 tok/s for the initial Windows cuTile path (4.62x). After cuTile KV-store repair and hybrid prefill dispatch, the eager-hybrid path reaches 557.66 tok/s (+20.4%); CUDA Graph replay is functionally repaired but performance-negative. For fixed-context agent workloads, warm prefix KV-cache reuse reduces computed prefill tokens to 64 and TTFT by 41.1%, 66.1%, and 79.7% for 1024/2048/3072-token prefixes, validated by a changed-prefix negative control (0% hit). CUDA Green Contexts, after activation validation, significantly smooth protected decode latency (sign test $p=0.0004$) but do not eliminate sequential prefill pauses. These results support teaching implementation, measurement, positive optimizations, and negative results in the full serving loop.

Keywords: LLM serving, CUDA Python, cuTile, hybrid prefill dispatch, prefix KV cache, CUDA Green Contexts

1. 서론

로컬 LLM 추론은 강의실, 연구실, 사적 데이터 분석 환경, 오픈라인 개발 환경에서 점점 중요해지고 있다. 이러한 환경에서 학습자가 필요로 하는 것은 단순히 LLM API를 호출하는 법이 아니라, 프롬프트가 어떻게 프리필(fill) 연산이 되는지, KV 캐시 블록이 어떻게 할당되는지, 디코드(decode)가 왜 지연에 민감한지, 스케줄러 결정이 체감 응답 시간에 어떻게 반영되는지, 그리고 어떤 최적화가 전체 서버 루프 안에서 왜 실행되는지를 이해하는 것을 이해하는 능력이다. vLLM, TensorRT-LLM, FlashAttention 기반 스레드, Triton 커널 등 다양한 추론 엔진의 서버 시스템은 높은 성능을 제공하지만 첫 교육용 아티팩트로는 적합하지 않다. 이들은 규모가 크고, 리눅스 중심이며, 최적화된 의존성과 긴밀하게 결합되어 있다[1,2,3]. WSL2는 Windows 사용자에게 실용적인 경로이지만, 학습자와 네이티브 런타임 사이에 또 하나의 경계를 추가한다. 많은 학생과 로컬 연구자가 NVIDIA GPU를 탑재한 소비자용 Windows 머신을 사용한다는 점에서, Windows 네이티브 GPU 개발은 여전히 중요하다. 본 논문은 이와 다른 기여를 목표로 한다. 바로 **교육용 시스템 아티팩트로서의 Windows 네이티브 LLM 서버 구현 및 성능 분석 사례 연구**이다. 마이크로-vLLM은 vLLM의 더 빠른 대체재로 위치하지 않는다. 학습자와 연구자가 검사해야 할 서버 메커니즘, 즉 프리필, 디코드, 페이징된 KV 캐시, 프리픽스 재사용, CUDA 컨텍스트 활성화, 스케줄러 동작, 할당 오버헤드, 꼬리 지연의 트레이드오프를 노출하도록 설계된다. 이 아티팩트는 서버 동작을 생산 스택 뒤에 숨기지 않고 **재현 가능한 근거**로 바꿀 때 **관심을 가진다**. 본 논문의 중심 워크로드는 고정 컨텍스트 에이전트 워크로드이다. 많은 에이전트 시스템은 짧은 사용자 요청 앞에 안정적인 컨텍스트를 반복해서 덧붙인다. 시스템 프롬프트, 데이터베이스 스키마, 도구 계약, **SKILL.md** 파일, 정책 텍스트, 예시, 루브릭, 강의 모듈이 그 예이다. Text-to-SQL 에이전트는 데이터베이스 스키마와 규칙 컨텍스트를 반복 포함하고, CUDA 튜터링 에이전트는 강의 자료와 커널 템플릿을 반복 포함하며, 교육용 지식 에이전트는 선택된 지식 번들 발췌를 반복 포함한다. 이러한 워크로드들은 비용이 큰 절적 프리픽스를 요청 강에 재사용할 수 있는 것으로 자연스레 프리픽스 KV 캐시 재사용에 적합하다. 코드 구성은 마이크로-vLLM 아티팩트는 0-MatMul, 1-FMA, 2-LLM와 from scratch, 3-micro-vLLM의 단계적 마이그레이션 경로로 구성된다. 본 논문의 벤치마크 구동기는 **bench_prefix_cache.py**와 **bench_green_stress.py**이다. 저장소는 공개되어 있다: [CtÉ Çu°... ÅiÀ~¹] **실습은 Windows 11, NVIDIA GeForce RTX 5090 (48 SM, 12GB VRAM, 48MB L2), PyTorch 2.13.0+cu130, CUDA 13.0, Qwen3-0.6B에서 수행되었다.**

2. 이론적 배경

2.1 LLM 서빙 루프

자기회귀(autoregressive) LLM 서빙은 프리필 단계와 디코드단계로 구성된다. 프리필은 프롬프트를 처리하여 모든 프롬프트 토큰에 대한 KV 캐시 엔트리를 기록한다. 디코드는 이전에 캐시된 키·밸류를 참조하면서 한 번에 하나의 토큰을 생성한다. 프리필은 프롬프트 길이와 병렬 연산 처리량에 민감하고, 디코드는 토큰당 지연, 메모리 대역폭, 스케줄러 오버헤드, KV 캐시 접근, ~~이 부분은 최적화 가능한 단계만 개선하면서도 중단 간 서비스 지표는 그대로 두는 이유를 설명한다.~~ ~~예컨대 프리픽스 캐시는 프리필 연산과 TTFT(Time-To-First-Token)를 크게 줄일 수 있지만, 벤치마크가 많은 출력 토큰을 생성하여 디코드가 총 실행 시간을 지배하면 처리량 개선은 제한적이다.~~

2.2 페이징된 KV 캐시와 프리픽스 재사용

페이징된 KV 캐시는 KV 메모리를 할당·매핑·재사용·추출 가능한 블록으로 나눈다[1]. 프리픽스 캐싱은 이 개념을 토큰-동일 프리픽스 재사용으로 확장한다. 새 요청이 이전 요청과 동일한 토큰 프리픽스를 공유하면, 런타임은 기존 KV 블록을 재사용하여 ~~저렴한 비용으로 계산할 수 있다~~ [6]. 안정성에 의존한다. 타임스탬프, 런 ID, 무작위 예시, 사용자별 텍스트처럼 동적인 메타데이터가 프롬프트 앞부분에 놓이면 재사용이 깨진다. 따라서 프롬프트 배치는 모델링 문제일 뿐 아니라 런타임 효율 문제이기도 하다. 프리픽스 재사용을 원한다면 정적 컨텍스트가 동적 사용자 콘텐츠보다 앞에 와야 한다.

2.3 CUDA Python, cuTile, nano-vLLM

CUDA Python은 CUDA 런타임·드라이버 API에 대한 Python 수준 접근을 제공하여, 호스트 측 GPU 제어를 가르치기에 유용하다[4]. cuTile 스타일 워크플로는 타일드 GPU 커널을 Python 안에서 표현함으로써 C++ CUDA보다 진입 장벽을 낮추면서도, 고수준 PyTorch 연산자보다 많은 하드웨어 동작을 노출한다. nano-VLLM은 vLLM 스타일 서빙을 위한 소형 참조 구현이다[8]. 마이크로-vLLM은 이러한 교육적 관점을 Windows 네이티브 CUDA Python 실험 경로에 적용한다.

2.4 CUDA 그린 컨텍스트

CUDA 그린 컨텍스트는 SM을 비롯한 GPU 자원을 컨텍스트 간에 분할할 수 있게 한다[5,7]. 본 논문에서 그린 컨텍스트는 일반적인 가속 메커니즘이 아니라 **자원 격리 메커니즘**으로 다룬다. 관심 문제는 프리필 간섭 하에서 지연에 민감한 디코드 경로를 보호할 수 있는가이다. 그린 컨텍스트 결과는 요청한 자원 분할 경로가 실제로 사용되었음을 활성화 메타데이터가 확인할 때만 유효하다.

3. 시스템 설계

3.1 설계 목표

마이크로-vLLM은 네 가지 목표를 중심으로 설계된다.
검사 가능성(inspectability): 학습자가 프롬프트 토큰이 어떻게 프리필 연산, KV 블록, 디코드 스텝이 되는지 추적할 수 있다.
수정 가능성(modifiability): 커널, 스케줄러 로직, 런타임 계층은 실험 중에 변경할 수 있을 만큼 작다.
Windows 네이티브 실행: 아티팩트는 WSL2를 유일한 경로로 삼지 않고 Windows CUDA 환경에서 실행된다.
에이전트 워크로드 연관성: 벤치마크는 교육용 에이전트, Text-to-SQL 에이전트, 로컬 지식 에이전트에서 반복되는 정적 컨텍스트를 반영한다.

3.2 마이크로-vLLM 아키텍처

단계	폴더	교육적 역할
0	0-MatMul	타일링, 공유 메모리, 스위칭, GEMM 기초 학습
1	1-FMHA	온라인 소프트맥스, 인과 마스크, 융합 어텐션 구현
2	2-LLM-from-scratch	최소 자기회귀 루프, KV 캐시, CUDA 그래프 실험
3	3-micro-vllm	페이징된 KV 캐시, 프리픽스 캐시, 연속 배치, 그린 컨텍스트

이 구조는 추론 엔진을 하나의 모놀리식 서빙 블

3.3 마이크로-vLLM 서빙 아키텍처

마이크로-vLLM은 경량 Qwen 계열 서빙 루프를 사용한다. 스케줄러는 대기 중인 시퀀스와 실행 중인 시퀀스를 관리한다. 블록 관리자는 KV 캐시 블록을 할당하고 해시 기반 프리픽스 재사용을 수행한다. 각 시퀀스는 프롬프트 토큰, 생성 토큰, 블록 테이블, 캐시된 토큰 수를 추적한다. 모델 러너는 입력 ID, 위치, 슬롯 매핑, 시퀀스 길이, 블록 테이블을 포함하여 프리필과 디코

동적 캐시를 분리한다. 동적 캐시의 경우, 스케줄러는 시퀀스 할당 시 완전한 블록 해시를 확인한다. 동일한 프리픽스 블록이 존재하면 seq.num_cached_tokens가 증가하고, 프리필 경로는 캐시되지 않은 접미사만 모델에 전달한다. 어텐션 컨텍스트는 여전히 전체 토큰 길이를 반영하므로, 모델은 계산된 프리픽스 KV 블록과 새로 계산된 접미사 KV 블록 모두를 참조할 수 있다. 그런 컨텍스트의 경우, 런타임은 요청한 정보가 실제로 활용되었는지를 기록한다. PyTorch GreenContext는 해당 환경에서 임포트는 가능했으나 컨텍스트 생성을 거부했다. 동작하는 경로는 cuda.core와 cuda.bindings.driver를 사용하여 Device.set_current(ctx)와 Device.set_current() (복원)를 호출하는 방식이었다. 벤치마크 JSON은 green_enabled, green_api_type, green_split_layout_width, green_prefill_resource_source를 기록하여, 활성화가 폴백(fallback)으로 떨어졌을 때 허위 성능 주장을 방지한다.

3.4 고정 컨텍스트 프롬프트 레이아웃

```
[È È ÅÄàÑ\ Ö , lÖ Ò, ]
[È È È ÌE / ³Ä-1 ¬ÄÄ}]
[È È DB ÅÐÐ¹È ¶ ²" ¬ ÇX îèÑMÂÐð, ]
[È È Æ ÄÜ ¶ ²" , è¼ ¹-]
[³ÜÈ Ä¬Æ©Ç• ÉÈ»8]
```

정적 프리픽스는 요청 간에 의도적으로 동일하고, 동적 접미사는 사용자 질문마다 달라진다. 이는 고정 SQLite 스키마 위의 Text-to-SQL, 고정 모듈 위의 CUDA 튜터링, 고정 소스 발췌 위의 nano-VLLM 튜터링, 안정적인 강의·지식 번들 컨텍스트 위의 검색 에이전트를 대표한다.

4. 실험 방법

4.1 하드웨어 및 소프트웨어

실험은 Windows 11 대상 PC에서 NVIDIA GeForce RTX 5070 GPU로 수행되었다. 로컬 아티팩트 메타데이터는 48 SM, 약 12GB VRAM, 48MB L2 캐시를 보고한다. 최근 그린 컨텍스트 프리플라이트 출력은 PyTorch 2.13.0+cu130과 CUDA 13.0을 보고한다. 마이크로-vLLM은 Qwen 게일 로컬 모델을 사용하며, 스트레스 벤치마크 로그는 대상 PC에서 Qwen3-0.6B를 식별한다.

4.2 매트릭 정의

본 논문의 성능 지표는 다음과 같이 정의한다. 처리량(tok/s)은 총 생성 토큰 수를 벽시계 시간(wall-clock time)으로 나눈 값이다. TTFT는 요청 도착부터 첫 생성 토큰까지의 지연이다. ITL은 토큰 간 디코드 지연(inter-token latency)이다. "연산 프리필 토큰"은 프리픽스 재사용 이후 새로 계산된 토큰만을 센다.

4.3 기준 처리량 벤치마크

기준 처리량 벤치마크는 동일한 동적 다중 사용자 벤치마크에서 Windows 네이티브 cuTile 경로를 WSL2 FlashAttention 기준 경로와 비교한다. 이 비교는 Windows cuTile의 우월성을 보이려는 것이 아니라, 성숙한 최적화 스택에 대비해 아티팩트를 고정(anchor)하기 위한 것이다.

4.4 프리픽스 캐시 벤치마크

프리픽스 캐시 벤치마크는 bench_prefix_cache.py를 사용한다. 각 정적 프리픽스 길이에 대해 세 가지 경우를 실행한다.	
no_cache	각 요청 전에 영속 해시 테이블을 비우고 전체 프리필 수행
warm_cache	캐시를 프라이밍한 뒤 동일 정적 프리픽스 재사용
prefix_changed	프리픽스를 변경하여 정확-프리픽스 캐시 히트를 붕괴

정적 프리픽스 길이는 1024, 2048, 3072 토큰이고, 동적 접미사는 64 토큰이다. 결과는 캐시된 토큰 수, 연산 프리필 토큰, TTFT, 종단 간 지연, 디코드 ITL, 처리량으로 표시된다.

4.5 그린 컨텍스트 스트레스 벤치마크

그린 컨텍스트 스트레스 벤치마크는 bench_green_stress.py를 사용한다. 하나의 보호 디코드 요청을 활성 상태로 유지하면서 큰 프리필 요청을 반복 주입한다. 주요 지표는 **보호 디코드 완료 간격(completion gap)**이며, 이는 현재 순차 엔진 루프에서 감시되었던 디코드 토큰 사이에 삽입되는 프리필 유발 일시정지를 포함한다.

벤치마크는 bench_green_stress.py를 사용한다. 보호 디코드 요청은 32 토큰 길이로 고정된다.	
보호 디코드 프롬프트	32 토큰
보호 디코드 출력	256 토큰
간섭 프리필 프롬프트	3072 토큰
간섭 프리필 출력	1 토큰

프리필 주입 횟수	12
주입 주기	4 디코드 스텝 이후, 매 8 디코드 스텝마다
그린 분할	프리필 32 SM, 디코드 16 SM

그린 측 서브프로세스는 유효한 개입으로 인정받기 위해 `green_enable`를 `1`로 설정하고, 디코딩 시 `green_enable`가 `1`인 경우에만 개입해야 한다.

5. 결과

5.1 WSL2 FlashAttention 기준 대 Windows cuTile

백엔드	실행 수	평균 시간	평균 처리량	상대 처리량
Windows cuTile	3	289.16 s	463.35 tok/s	1.00x
WSL2 FlashAttention	4	62.65 s	2138.55 tok/s	4.62x

이 결과는 초기 Windows 네이티브 cuTile의 약점이 아니며, 기여를 생산하는 데 도움이 됩니다.

5.2 하이브리드 프리필 디스패치와 CUDA 그래프 재생

초기 cuTile 기준 이후, PyTorch 고급 인덱싱 기반 KV 캐시 갱신을 cuTile KV-스토어 커널로 교체하여 디코드 그래프 캡처 경로를 수리했다. 또한 하이브리드 프리필 디스패치를 도입했다. 다중 시퀀스 비-프리픽스 프리필은 배치 패딩 래퍼를 사용해 호스트/커널 실행 횟수를 줄이고, 단일 시퀀스와 프리픽스 캐시 프리필은 불필요한 텐서 실체화를 피하기 위해 직접(direct) 또는 eager-hybrid cuTile 커널을 사용한다.

cuTile 모드	그래프	프리필 전략	총 시간	처리량	해석
초기 cuTile 평균	off	eager 기준	289.16 s	463.35 tok/s	초기 Windows cuTile 기준
Hybrid eager	off	hybrid	240.23 s	557.66 tok/s	현재 논문의 긍정적 cuTile 결과
Hybrid graph	on	hybrid	340.60 s	393.32 tok/s	캡처 성공, 처리량 하락
Padded graph	on	padded	341.02 s	392.84 tok/s	그래프 재생이 주 병목임을 확인
Direct graph	on	direct	조기 중단	95 – 101 tok/s	256-요청 벤치마크에 부적합

eager-hybrid cuTile 경로는 여전히 실패했지만 현재 성능은 하락하지 않았으며, 고정된 디코드 메모리 접근을 허용한다.

CUDA 그래프 실패 원인 분석

CUDA 그래프 결과는 두 가지 별개의 실패로 해석해야 한다. 첫 번째는 **활성화 실패**이다. 초기 cuTile 디코드 경로는 `slot_mapping[mask], k_cache_flat[valid_slots] = ...`와 같은 PyTorch 부울 마스킹 및 고급 인덱싱으로 KV 캐시 엔트리를 갱신했다. 그 경로는 캡처 중 호스트 런타임과 동적 인덱싱 동작을 호출하여 `cudaErrorStreamCaptureInvalidated` 실패를 일으켰다. 이를 고정 cuTile `store_kvcache_cutile_kernel`로 교체하여 그래프 캡처를 수리했다. 두 번째는 **성능 실패**이다. 캡처가 성공한 뒤에도 현재 cuTile 디코드 그래프는 동적 서빙 워크로드와 아직 정합하지 않는다. 그래프 버킷은 활성 토큰 집합이 요구하는 것보다 큰 고정 형태를 재생하여 과잉 계산을 일으킬 수 있다. 디코드 경로는 작은 커널이 많아 실행 제거의 이점이 커널 세분성에 의해 제한된다. 페이지징된 KV 캐시 디코드는 불규칙한 메모리 접근을 보존하므로, 그래프 재생만으로는 그 메모리 병목이 제거되지 않는다. 직접 그래프-프리필 모드는 다중 요청 프리필에서 시퀀스당 실행 압력을 추가로 높여 256-요청 벤치마크에서 부적합해졌다. 따라서 본 논문에서 CUDA 그래프는 기능적으로 사용 가능해졌으나 아직 가속으로 검증되지 않은 최적화로 보고한다.

5.3 고정 컨텍스트 워크로드를 위한 프리픽스 KV 캐시

정적 프리픽스	프롬프트 토큰	유효 히트율	프리필 no-cache	프리필 warm	TTFT no-cache	TTFT warm	TTFT 감소율	프리픽스 변경 TTFT
1024	1088	94.1%	1088	64	146.98 ms	86.56 ms	41.1%	148.18 ms

2048	2112	97.0%	2112	64	255.58 ms	86.72 ms	66.1%	256.93 ms
3072	3136	98.0%	3136	64	381.76 ms	77.47 ms	79.7%	389.61 ms

임 프리픽스 캐시는 재사용되고, 64토큰 프리픽스 변경 비용은 41.1% (2048 토큰에 정해진 토큰 풀링을 통한 처리량)을 지배하기 때문으로 방향성이 아니다.

5.4 부정적 결과로서의 동적 형태 패딩

동적 형태 패딩은 JIT 형태 변이를 줄이기 위한 것이었다. 그러나 실제로는 각 디코드 스텝마다 임시 텐서 할당과 복사 오버헤드를 일으켜 PyTorch CUDA 캐싱 할당자 압력을 높였다. 무패딩 eager 경로는 470.82 tok/s를 달성한 반면, 패딩 경로는 155.16 tok/s로 67.04%의 처리량 감소를 보였다. 이는 부정적 결과는 교육적으로도 중요하다. 지역적 최적화 가설은 할당 동작과 호스트 측 런타임 오버헤드를 포함한 전체 서빙 루프 안에서 평가되어야 함을 보여준다.

5.5 그린 컨텍스트 활성화와 비스트레스 결과

초기 그린 컨텍스트 로그는 요청한 그린 경로가 조용히 폴백했기 때문에 유효한 효능 측정이 아니었다. 이후 런타임에 활성화 메타데이터를 계측했다. 유효한 cuda-core 실행은 32/16 SM 분할과 green_prefill_resource_source="device_sm_fallback"으로 20/20 그린 측 실행에서 green_enabled=true와 green_api_type="cuda_core"를 기록했다. 비스트레스 페어 벤치마크에서 유효한 훈련 실행은 기준선 P99 이상치 하나를 제거한 뒤에는 안정적인 서빙 수준 이점을 보이지 않았다. 그 이상치를 제외하면 TTFT는 평균 +0.49%, P99 ITL은 +0.32%, 처리량은 +2.06% 변했다. 이는 일반적인 가속 주장이 아니라 스트레스 워크로드를 동기로 삼게 한다.

5.6 그린 컨텍스트 스트레스 결과

비스트레스는 2072토큰 풀링을 개선하는데 1.1%의 그린 컨텍스트는 비스트레스 기준선 이상의 평균치를 보인다.

지표	기준 평균	그린 평균	평균 델타	중앙값 델타	개선 run
디코드 스텝 P50	50.56 ms	47.37 ms	□5.38%	□5.97%	14/20
디코드 스텝 P95	66.33 ms	62.89 ms	□4.83%	□5.41%	13/20
디코드 스텝 P99	71.56 ms	68.42 ms	□4.30%	□3.39%	18/20
디코드 간격 P50	51.03 ms	47.89 ms	□5.28%	□5.39%	14/20
디코드 간격 P95	79.45 ms	75.33 ms	□4.98%	□5.77%	16/20
디코드 간격 P99	432.31 ms	426.35 ms	□1.29%	□1.15%	13/20
디코드 간격 최대	441.38 ms	437.94 ms	□0.70%	□0.87%	14/20
처리량	2098.17 tok/s	2188.79 tok/s	+4.75%	+5.19%	14/20

20회 페어 실행에 대한 양측 검정 P99, 18/20, p=0.0004; 위차간값은 각각 14/20, 16/20, 18/20 페어 실행에서 개선이 가장 많이 남는다. 이는 현재 기준선보다. 따라서 그린 컨텍스트는

6. 고찰

6.1 고정 컨텍스트 에이전트가 중요한 이유

에이전트 워크로드는 논리적으로 상수인 컨텍스트에 대해 프리필 비용을 반복 지불한다. Text-to-SQL 에이전트는 스키마와 규칙 컨텍스트를, CUDA 튜터는 강의 자료·코드 스니펫·루브릭을, 지식 에이전트는 선택된 지식 번들 콘텐츠를 반복 포함한다. 프리픽스 KV 캐시는 이러한 고정 컨텍스트를 런타임에 가시화한다. 안정적 컨텍스트는 프롬프트 앞에 배치하고, 휘발성 메타데이터는 뒤로 미루거나 캐시된 프리픽스에서 제외해야 한다. 교육용·데이터 질의 에이전트의 경우, 런타임을 인식한 프롬프트 배치는 모델 변경 없이 TTFT를 줄일 수 있다.

6.2 그린 컨텍스트 결과가 가르치는 교훈

그린 컨텍스트는 다른 교훈을 가르친다. 이는 일반 가속기가 아니라 자원 분할 기반(substrate)이다. 아티팩트는 세 가지 엔지니어링 및 운영을 보여준다. 첫째, 활성화 유효성을 측정해야 한다. 초기 그린 실행은 런타임이 조용히 폴백했기 때문에 무효였다. 둘째, 서빙 루프가 순차적이면 자원 분할만으로 지연 개선이 보장되지 않는다. 셋째, 반복 프리필 간섭이 있는 스트레스 워크로드에서 그린 컨텍스트는 보호 디코드 작업을 완만하게 평활화하지만, 프리필 유발 일시정지를 제거하지는 못한다. 이는 GPU 기동에 의한 분할을 뒤집지하기 전에, 프리필이 실제로 활성화되었는지와 스케줄러가 올바른 간섭 패턴을 만드는지를 증명해야 한다는 흔한 실수를 예방하므로 유용한 교육적 결과다.

6.3 부정적 결과의 교육적 가치

본 논문은 부정적·제한적 결과를 의도적으로 포함한다. WSL2 FlashAttention은 초기 Windows cuTile 경로보다 훨씬 빠르다. 하이브리드 eager는 cuTile 경로를 개선하지만, CUDA 그래프 재생은 현재 그래프 버킷과 페이징 디코드 조건에서 처리량을 해친다. 동적 패딩은 처리량을 해친다. 그린 컨텍스트는 활성화 메타데이터를 요구하며 현재 엔진에서 제한적 이점만 보인다. 이러한 결과는 시스템 주장이 어떻게 형성되어야 하는지, 즉 메커니즘, 계측, 대조 조건, 측정, 보수적 해석의 순서를 보여줌으로써 아티팩트를 교육에 더 유용하게 만든다.

6.4 제출 범위

본 논문은 더 넓은 에이전트 프레임워크 기여를 포함하지 않는다. 그러한 주제는 별도의 교육용 지식 에이전트 논문에 적합하다. 본 논문은 추론 엔진 아티팩트로서의 마이크로-vLLM과 측정된 서빙 루프 동작에 초점을 유지한다.

7. 결론

본 논문은 마이크로-vLLM을 Windows 네이티브 LLM 서빙 시스템 구현 및 성능 분석 사례연구로 제시한다. 아티팩트는 성숙한 리눅스 서빙 스택을 능가하지 않는다. WSL2 FlashAttention은 초기 Windows cuTile 백엔드 대비 평균 4.62배 높은 처리량을 달성한다. 그러나 cuTile KV-스토어 수리와 하이브리드 프리필 디스패치 이후, eager-hybrid cuTile 경로는 초기 cuTile 평균 대비 20.4% 향상된 557.66 tok/s에 도달한다. 따라서 마이크로-vLLM은 학생과 연구자가 완전한 서빙 루프 인과성을 관찰할 수 있는 가장 강력하게 특성화 가능한 마이그레이션 및 실험 경로를 제공한다. 워밍업은 모든 정적 프리픽스 길이에서 연산 프리필 토큰을 64로 줄이고, 1024·2048·3072토큰 프리픽스에 대해 TTFT를 각각 41.1%, 66.1%, 79.7% 감소시킨다. 프리픽스 변경은 예상대로는 정적 프리픽스 의존성을 검증한다. 그린 컨텍스트 실행은 제한된 시스템 통찰을 제공한다. cuda-core 기반 컨텍스트는 RTX 5070 대상 PC에서 성공적으로 활성화되고, 반복 프리필 간섭 하에서 보호 디코드 지연을 완만하게 평활화한다. 디코드 스텝 P99는 평균 4.3% 개선되고 20회 페어 스트레스 실행 중 18회에서 개선된다(부호검정 p=0.0004). 그러나 디코드 간격 P99와 최대 간격은 순차 프리필 삽입에 지배된 채 남아, 비동기 서빙 루프 없이는 SM 분할만으로 충분하지 않음을 보여준다. 종합하면, 이 결과는 본 논문의 교육적 부가점을 뒷받침한다. 효과적인 추론 엔진 교육은 성공적 최적화뿐 아니라 구현 트레이드오프, 활성화 유효성, 부정적 결과, 워크로드 의존성, 그리고 고립된 커널 동작과 종단 간 서빙 동작의 구분까지 가르쳐야 한다.

참고문헌

1. W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP '23), pp. 611–626, 2023.

2. T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Riquelme, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in Advances in Neural Information Processing Systems (NeurIPS), vol. 35, pp. 16344–16359, 2022.

3. P. Tillet, H.-T. Kung, and D. Cox, "Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations," in Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL '19), pp. 10–19, 2019.

4. NVIDIA Corporation, "CUDA Python Documentation," NVIDIA Developer Documentation. [Online]. Available: <https://nvidia.github.io/cuda-python/> (Accessed: 2025).

5. NVIDIA Corporation, "Green Contexts," in CUDA C++ Programming Guide, NVIDIA Developer Documentation. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/> (Accessed: 2025).

6. vLLM Project, "Automatic Prefix Caching," vLLM Documentation. [Online]. Available: https://docs.vllm.ai/en/latest/automatic_prefix_caching/apc.html (Accessed: 2025).

7. PyTorch Foundation, "torch.cuda.green_contexts: Granular Resource Partitioning for CUDA Kernels," PyTorch Documentation. [Online]. Available: <https://pytorch.org/docs/stable/> (Accessed: 2025).

8. GeeekExplorer, "nano-VLLM," GitHub repository. [Online]. Available: <https://github.com/GeeekExplorer/nano-vLLM> (Accessed: 2025).

9. "KernelAgent," micro-vLLM artifact repository, GitHub repository. [이중 익명 심사를 위해 저장소 URL 생략].

부록

부록 A.1 프리픽스 캐시 벤치마크 조건

--	--

no_cache	각 요청 전에 영속 해시 테이블을 비우고 전체 프리필 수행
warm_cache	캐시를 프라이밍한 뒤 동일 정적 프리픽스 재사용
prefix_changed	프리픽스를 변경하여 정확-프리픽스 캐시 히트를 붕괴

부록 A.2 그린 컨텍스트 스트레스 워크로드

매개변수	값
보호 디코드 프롬프트	32 토큰
보호 디코드 출력	256 토큰
간접 프리필 프롬프트	3072 토큰
간접 프리필 출력	1 토큰
프리필 주입 횟수	12
주입 주기	4 디코드 스텝 이후, 매 8 디코드 스텝마다
그린 분할	프리필 32 SM, 디코드 16 SM

